

PHP

This article is about the scripting language. For other uses, see PHP (disambiguation).

PHP

	
Paradigm(s)	imperative, functional, object-oriented, procedural, reflective
Designed by	Rasmus Lerdorf
Developer	The PHP Group
Appeared in	1995
Stable release	5.5.13 / May 29, 2014
Preview release	5.6.0beta4 / June 5, 2014
Typing discipline	Dynamic, weak
Major implementations	Zend Engine, Phalanger, Quercus, Project Zero, HipHop, HHVM, Parrot
Influenced by	Perl, C, C++, Java, Tcl
Influenced	Hack
Implementation language	C
OS	Cross-platform
License	PHP License
Filename extension(s)	.php, .phtml, .php4, .php3, .php5, .phps
Website	www.php.net ^[1]
•  PHP Programming at Wikibooks	

PHP is a server-side scripting language designed for web development but also used as a general-purpose programming language. As of January 2013^[2], PHP was installed on more than 240 million websites (39% of those sampled) and 2.1 million web servers. Originally created by Rasmus Lerdorf in 1994, the reference implementation of PHP is now produced by The PHP Group. While PHP originally stood for *Personal Home Page*, it now stands for *PHP: Hypertext Preprocessor*, a recursive backronym.^[3]

PHP code can be simply mixed with HTML code, or it can be used in combination with various templating engines and web frameworks. PHP code is usually processed by a PHP interpreter, which is usually implemented as a web server's native module or a Common Gateway Interface (CGI) executable. After the PHP code is interpreted and executed, the web server sends resulting output to its client, usually in form of a part of the generated web page – for example, PHP code can generate a web page's HTML code, an image, or some other data. PHP has also evolved to include a command-line interface (CLI) capability and can be used in standalone graphical applications.

PHP is free software released under the PHP License. PHP has been widely ported and can be deployed on most web servers on almost every operating system and platform, free of charge.

History



Rasmus Lerdorf, who wrote the original Common Gateway Interface (CGI) component, together with Andi Gutmans and Zeev Suraski, who rewrote the parser that formed PHP 3.

PHP development began in 1994 when the developer Rasmus Lerdorf wrote a series of Common Gateway Interface (CGI) Perl scripts, which he used to maintain his personal homepage. The tools performed tasks such as displaying his résumé and recording his web traffic. He rewrote these scripts in C for performance reasons, extending them to add the ability to work with web forms and to communicate with databases, and called this implementation "Personal Home Page/Forms Interpreter" or PHP/FI.

PHP/FI could be used to build simple, dynamic web applications. Lerdorf initially announced the release of PHP/FI as "Personal Home Page Tools (PHP Tools) version 1.0" publicly to accelerate bug location and improve the code, on the Usenet discussion group *comp.infosystems.www.authoring.cgi* on June 8, 1995. This release already had the basic functionality that PHP has as of 2013[2]. This included Perl-like variables, form handling, and the ability to embed HTML. The syntax resembled that of Perl but was simpler, more limited and less consistent.

Early PHP was not intended to be a new programming language, and grew organically, with Lerdorf noting in retrospect: "I don't know how to stop it, there was never any intent to write a programming language [...] I have absolutely no idea how to write a programming language, I just kept adding the next logical step on the way."^[4] A development team began to form and, after months of work and beta testing, officially released PHP/FI 2 in November 1997.

One criticism of PHP is that it was not originally designed, but instead it was developed organically; among other things, this has led to inconsistent naming of functions and inconsistent ordering of their parameters. In some cases, the function names were chosen to match the lower-level libraries which PHP was "wrapping", while in some very early versions of PHP the length of the function names was used internally as a hash function, so names were chosen to improve the distribution of hash values.

Zeev Suraski and Andi Gutmans rewrote the parser in 1997 and formed the base of PHP 3, changing the language's name to the recursive acronym *PHP: Hypertext Preprocessor*. Afterwards, public testing of PHP 3 began, and the official launch came in June 1998. Suraski and Gutmans then started a new rewrite of PHP's core, producing the Zend Engine in 1999. They also founded Zend Technologies in Ramat Gan, Israel.

On May 22, 2000, PHP 4, powered by the Zend Engine 1.0, was released. As of August 2008 this branch reached version 4.4.9. PHP 4 is no longer under development nor will any security updates be released.

On July 13, 2004, PHP 5 was released, powered by the new Zend Engine II. PHP 5 included new features such as improved support for object-oriented programming, the PHP Data Objects (PDO) extension (which defines a lightweight and consistent interface for accessing databases), and numerous performance enhancements. In 2008 PHP 5 became the only stable version under development. Late static binding had been missing from PHP and was added in version 5.3.

Many high-profile open-source projects ceased to support PHP 4 in new code as of February 5, 2008, because of the GoPHP5 initiative, provided by a consortium of PHP developers promoting the transition from PHP 4 to PHP 5.

PHP interpreters are available on most existing 32-bit and 64-bit operating systems, either by building them from the PHP source code, or by using pre-built binaries. For the PHP versions 5.3 and 5.4, the only available Microsoft Windows binary distributions were 32-bit x86 builds, requiring Windows 32-bit compatibility mode while using Internet Information Services (IIS) on a 64-bit Windows platform. PHP version 5.5 made the 64-bit x86-64 builds available for Microsoft Windows.

PHP 6 and Unicode

PHP received mixed reviews due to lacking native Unicode support at the core language level. In 2005, a project headed by Andrei Zmievski was initiated to bring native Unicode support throughout PHP, by embedding the International Components for Unicode (ICU) library, and representing text strings as UTF-16 internally. Since this would cause major changes both to the internals of the language and to user code, it was planned to release this as version 6.0 of the language, along with other major features then in development.

However, a shortage of developers who understood the necessary changes, and performance problems arising from conversion to and from UTF-16, which is rarely used in a web context, led to delays in the project. As a result, a PHP 5.3 release was created in 2009, with many non-Unicode features back-ported from PHP 6, notably namespaces. In March 2010, the project in its current form was officially abandoned, and a PHP 5.4 release was prepared containing most remaining non-Unicode features from PHP 6, such as traits and closure re-binding. Initial hopes were that a new plan would be formed for Unicode integration, but as of 2014 none has been adopted.

During the years before the release of PHP 5.3 and 5.4, some books were published based on the expected feature set of PHP 6.0, including both the Unicode work and the features which were later backported to other releases. There is therefore some debate over whether a new major version of PHP, with or without Unicode support, should be called "PHP 6", or if the version should be skipped to avoid confusion.

Release history

Key

Color	Meaning	Development
Red	Old release	No development
Yellow	Stable release	Security fixes
Green	Stable release	Bug and security fixes
Blue	Future release	New features

Version	Release date	Supported until	Notes
1.0	8 June 1995		Officially called "Personal Home Page Tools (PHP Tools)". This is the first use of the name "PHP".
2.0	1 November 1997		
3.0	6 June 1998	20 October 2000	Development moves from one person to multiple developers. Zeev Suraski and Andi Gutmans rewrite the base for this version.
4.0	22 May 2000	23 January 2001	Added more advanced two-stage parse/execute tag-parsing system called the Zend engine.
4.1	10 December 2001	12 March 2002	Introduced 'superglobals' (<code>\$_GET</code> , <code>\$_POST</code> , <code>\$_SESSION</code> , etc.)
4.2	22 April 2002	6 September 2002	Disabled <code>register_globals</code> by default. Data received over the network is not inserted directly into the global namespace anymore, closing possible security holes in applications.
4.3	27 December 2002	31 March 2005	Introduced the command-line interface (CLI), to supplement the CGI.
4.4	11 July 2005	7 August 2008	Fixed a memory corruption bug, which required breaking binary compatibility with extensions compiled against PHP version 4.3.x.
5.0	13 July 2004	5 September 2005	Zend Engine II with a new object model.
5.1	24 November 2005	24 August 2006	Performance improvements with introduction of compiler variables in re-engineered PHP Engine. Added PHP Data Objects (PDO) as a consistent interface for accessing databases.
5.2	2 November 2006	6 January 2011	Enabled the filter extension by default. Native JSON support.
5.3	30 June 2009	July 2014	Namespace support; late static bindings, Jump label (limited goto), Native closures, Native PHP archives (phar), garbage collection for circular references, improved Windows support, sqlite3, mysqlnd as a replacement for libmysql as underlying library for the extensions that work with MySQL, fileinfo as a replacement for mime_magic for better MIME support, the Internationalization extension, and deprecation of ereg extension.
5.4	1 March 2012	1 March 2015	Trait support, short array syntax support. Removed items: <code>register_globals</code> , <code>safe_mode</code> , <code>allow_call_time_pass_reference</code> , <code>session_register()</code> , <code>session_unregister()</code> and <code>session_is_registered()</code> . Built-in web server. Several improvements to existing features, performance and reduced memory requirements.
5.5	20 June 2013	20 June 2016	Support for generators, <code>finally</code> blocks for exceptions handling, OpCache (based on Zend Optimizer+) bundled in official distribution.
5.6	No date set	3 years after release	Constant scalar expressions, variadic functions, argument unpacking, new exponentiation operator, extensions of the <code>use</code> operator, new <code>phpdbg</code> debugger as a SAPI module, and other smaller improvements.

Beginning on June 28, 2011, the PHP Group began following a timeline for when new versions of PHP will be released. Under this timeline, at least one release should occur every month. Once per year, a minor release should occur which can include new features. Every minor release should at least have 2 years of security and bug fixes, followed by at least 1 year of only security fixes, for a total of a 3 year release process for every minor release. No new features (unless small and self-contained) will be introduced into a minor release during the 3-year release

process.

Syntax

Main article: PHP syntax and semantics

The following Hello world program is written in PHP code embedded in an HTML document:

```
<!DOCTYPE html>
<html>
  <head>
    <title>PHP Test</title>
  </head>
  <body>
    <?php echo '<p>Hello World</p>'; ?>
  </body>
</html>
```

However, as PHP does not need to be embedded in HTML or used with a web server, the simplest version of a Hello World program can be written like this, with the closing tag omitted as preferred in files containing pure PHP code (prior to PHP 5.4.0, this short syntax for `echo()` only works with the `short_open_tag` configuration setting enabled, while for PHP 5.4.0 and later it is always available):

```
<?= 'Hello world';
```

The PHP interpreter only executes PHP code within its delimiters. Anything outside its delimiters is not processed by PHP (although non-PHP text is still subject to control structures described in PHP code). The most common delimiters are `<?php` to open and `?>` to close PHP sections. `<script language="php">` and `</script>` delimiters are also available, as are the shortened forms `<?` or `<?='` (which is used to echo back a string or variable) and `?>` as well as ASP-style short forms `<%` or `<%=` and `%>`. Short delimiters make script files less portable, since support for them can be disabled in the local PHP configuration, and they are therefore discouraged. The purpose of all these delimiters is to separate PHP code from non-PHP code, including HTML.

The first form of delimiters, `<?php` and `?>`, in XHTML and other XML documents, creates correctly formed XML "processing instructions". This means that the resulting mixture of PHP code and other markup in the server-side file is itself well-formed XML.

Variables are prefixed with a dollar symbol, and a type does not need to be specified in advance. PHP 5 introduced *type hinting* that allows functions to force their parameters to be objects of a specific class, arrays, interfaces or callback functions. However, type hints can not be used with scalar types such as integer or string.

Unlike function and class names, variable names are case sensitive. Both double-quoted ("") and heredoc strings provide the ability to interpolate a variable's value into the string. PHP treats newlines as whitespace in the manner of a free-form language (except when inside string quotes), and statements are terminated by a semicolon. PHP has three types of comment syntax: `/* */` marks block and inline comments; `//` as well as `#` are used for one-line comments. The `echo` statement is one of several facilities PHP provides to output text, *e.g.*, to a web browser.

In terms of keywords and language syntax, PHP is similar to most high level languages that follow the C style syntax. *if* conditions, *for* and *while* loops, and function returns are similar in syntax to languages such as C, C++, C#, Java and Perl.

Data types

PHP stores whole numbers in a platform-dependent range, either a 64-bit or 32-bit signed integer equivalent to the C-language long type. Unsigned integers are converted to signed values in certain situations; this behavior is different from other programming languages. Integer variables can be assigned using decimal (positive and negative), octal, hexadecimal, and binary notations.

Floating point numbers are also stored in a platform-specific range. They can be specified using floating point notation, or two forms of scientific notation. PHP has a native Boolean type that is similar to the native Boolean types in Java and C++. Using the Boolean type conversion rules, non-zero values are interpreted as true and zero as false, as in Perl and C++.

The null data type represents a variable that has no value; *NULL* is the only allowed value for this data type.

Variables of the "resource" type represent references to resources from external sources. These are typically created by functions from a particular extension, and can only be processed by functions from the same extension; examples include file, image, and database resources.

Arrays can contain elements of any type that PHP can handle, including resources, objects, and even other arrays. Order is preserved in lists of values and in hashes with both keys and values, and the two can be intermingled. PHP also supports strings, which can be used with single quotes, double quotes, nowdoc or heredoc syntax.

The Standard PHP Library (SPL) attempts to solve standard problems and implements efficient data access interfaces and classes.

Functions

PHP has hundreds of functions provided by the core language functionality and thousands more available via various extensions; these functions are well documented in the online PHP documentation. However, the built-in library has a wide variety of naming conventions and associated inconsistencies, as described under history above.

Additional functions can be defined by the developer:

```
function myAge($birthYear) //
defines a function, this one is named "myAge"
{
    $yearsOld = date('Y') - $birthYear; //
calculates the age
    return $yearsOld . ' year' . ($yearsOld != 1 ? 's' : ''); //
returns the age in a descriptive form
}

echo 'I am currently ' . myAge(1981) . ' old.'; //
outputs the text concatenated

//
with the return value of myAge()
// As the result of this syntax, myAge() is called.
// In 2014, the output of this sample program will be 'I am currently
33 years old.'
```

In PHP 5.2 and earlier, functions are not first-class functions and can only be referenced by their name, directly or dynamically by a variable containing the name of the function. User-defined functions can be created at any time without being prototyped. Functions can be defined inside code blocks, permitting a run-time decision as to whether

or not a function should be defined. Function calls must use parentheses, with the exception of zero-argument class constructor functions called with the `PHP new` operator, where parentheses are optional.

PHP supports quasi-anonymous functions through the `create_function()` function. However, they are not truly anonymous because they can be referenced either by name, or indirectly as variable functions using `$function_name()` syntax, as seen in the following example:

```
$f = create_function('$a, $b = 5', 'var_dump($a, $b);');
$f(1);

// output:
// int(1)
// int(5)
```

PHP 5.3 added support for closures, which enable true anonymous functions. The syntax can be seen in the following example:

```
function getAdder($x)
{
    return function($y) use ($x)
    {
        return $x + $y;
    };
}

$adder = getAdder(8);
echo $adder(2); // prints "10"
```

Here, the `getAdder()` function creates a closure using passed argument `$x` (the keyword `use` imports a variable from the lexical context), which takes an additional argument `$y`, and returns the created closure to the caller. Such a function is a first-class object, meaning that it can be stored in a variable, passed as a parameter to other functions, etc. For more details, see the [Lambda functions and closures RFC](#) ^[5].

The `goto` flow control statement is used as in the following example:

```
function lock()
{
    $file = fopen('file.txt', 'r+');

retry:
    if (!flock($file, LOCK_EX | LOCK_NB))
        goto retry;

    fwrite($file, 'Success!');
    fclose($file);
}
```

When `flock()` is called, PHP opens a file and tries to lock it. The target label `retry:` defines the point to which execution should return if `flock()` is unsuccessful and `goto retry;` is called. The `goto` statement is

restricted and requires that the target label be in the same file and context.

The `goto` statement has been supported since PHP 5.3.

Objects

Basic object-oriented programming functionality was added in PHP 3 and improved in PHP 4. Object handling was completely rewritten for PHP 5, expanding the feature set and enhancing performance. In previous versions of PHP, objects were handled like value types. The drawback of this method was that the whole object was copied when a variable was assigned or passed as a parameter to a method. In the new approach, objects are referenced by handle, and not by value.

PHP 5 introduced private and protected member variables and methods, along with abstract classes, final classes, abstract methods, and final methods. It also introduced a standard way of declaring constructors and destructors, similar to that of other object-oriented languages such as C++, and a standard exception handling model. Furthermore, PHP 5 added interfaces and allowed for multiple interfaces to be implemented. There are special interfaces that allow objects to interact with the runtime system. Objects implementing `ArrayAccess` can be used with array syntax and objects implementing `Iterator` or `IteratorAggregate` can be used with the `foreach` language construct. There is no virtual table feature in the engine, so static variables are bound with a name instead of a reference at compile time.

If the developer creates a copy of an object using the reserved word `clone`, the Zend engine will check whether a `__clone()` method has been defined. If not, it will call a default `__clone()` which will copy the object's properties. If a `__clone()` method is defined, then it will be responsible for setting the necessary properties in the created object. For convenience, the engine will supply a function that imports the properties of the source object, so the programmer can start with a by-value replica of the source object and only override properties that need to be changed.

The following is a basic example of object-oriented programming in PHP:

```
class Person
{
    public $firstName;
    public $lastName;

    public function __construct($firstName, $lastName = '') { //
optional second argument
        $this->firstName = $firstName;
        $this->lastName  = $lastName;
    }

    public function greet() {
        return 'Hello, my name is ' . $this->firstName .
            (($this->lastName != '') ? (' ' . $this->lastName) : '') .
            '.';
    }

    public static function staticGreet($firstName, $lastName) {
        return 'Hello, my name is ' . $firstName . ' ' . $lastName .
            '.';
    }
}
```



```
$he      = new Person('John', 'Smith');
$she     = new Person('Sally', 'Davis');
$other   = new Person('iAmine');

echo $he->greet(); // prints "Hello, my name is John Smith."
echo '<br />';

echo $she->greet(); // prints "Hello, my name is Sally Davis."
echo '<br />';

echo $other->greet(); // prints "Hello, my name is iAmine."
echo '<br />';

echo Person::staticGreet('Jane', 'Doe'); // prints "Hello, my name is
Jane Doe."
```

The visibility of PHP properties and methods is defined using the keywords `public`, `private`, and `protected`. The default is `public`, if only `var` is used; `var` is a synonym for `public`. Items declared `public` can be accessed everywhere. `protected` limits access to inherited classes (and to the class that defines the item). `private` limits visibility only to the class that defines the item. Objects of the same type have access to each other's `private` and `protected` members even though they are not the same instance. PHP's member visibility features have sometimes been described as "highly useful." However, they have also sometimes been described as "at best irrelevant and at worst positively harmful."

Implementations

PHP source code is compiled on-the-fly to an internal format that can be executed by the PHP engine, which works as an interpreter. In order to speed up execution time and not have to compile the PHP source code every time the web page is accessed, PHP scripts can also be deployed in the PHP engine's internal format by using a PHP compiler. Code optimizers aim to enhance the performance of the compiled code by reducing its size, merging redundant instructions and making other changes that can reduce the execution time. With PHP, there are often opportunities for code optimization. An example of a code optimizer is the eAccelerator PHP extension.

Another approach for reducing compilation overhead for PHP servers is using an opcode cache. Opcode caches work by caching the compiled form of a PHP script (opcodes) in shared memory to avoid the overhead of parsing and compiling the code every time the script runs. An opcode cache, Zend OpCache, is built into PHP since version 5.5. Another example of a widely used opcode cache is the Alternative PHP Cache (APC), which is available as a PECL extension.

The PHP language was originally implemented as an interpreter, and this is still the most popular implementation. Several compilers have been developed which decouple the PHP language from the interpreter. Advantages of compilation include better execution speed, static analysis, and improved interoperability with code written in other languages. Notable PHP compilers include the following:

- HipHop Virtual Machine (HHVM) – developed at Facebook and available as open source, it converts PHP code into a high-level bytecode (commonly known as an intermediate language), which is then translated into x86-64 machine code dynamically at runtime by a just-in-time (JIT) compiler, resulting in up to 6× performance improvements.
- Parrot – a virtual machine designed to run dynamic languages efficiently; Pipp transforms the PHP source code into the Parrot intermediate representation, which is then translated into the Parrot's bytecode and executed by the

virtual machine.

- Phalanger – compiles PHP into Common Intermediate Language (CIL) bytecode
- HipHop – developed at Facebook and available as open source, it transforms the PHP scripts into C++ code and then compiles the resulting code, reducing the server load up to 50%. In early 2013, Facebook deprecated it in favor of HHVM due to multiple reasons, including deployment difficulties and lack of support for the whole PHP language, including the `create_function()` and `eval()` constructs.

Licensing

PHP is free software released under the PHP License, which stipulates that:

Products derived from this software may not be called "PHP", nor may "PHP" appear in their name, without prior written permission from group@php.net. You may indicate that your software works in conjunction with PHP by saying "Foo for PHP" instead of calling it "PHP Foo" or "phpfoo".

This restriction on use of the name *PHP* makes it incompatible with the GNU General Public License (GPL).

Development and community

PHP includes free and open source libraries with the core build. PHP is a fundamentally Internet-aware system with modules built in for accessing File Transfer Protocol (FTP) servers, many database servers, embedded SQL libraries such as embedded PostgreSQL, MySQL, Microsoft SQL Server and SQLite, LDAP servers, and others. Many functions familiar to C programmers such as those in the stdio family are available in the standard PHP build.

PHP allows developers to write extensions in C to add functionality to the PHP language. These can then be compiled into PHP or loaded dynamically at runtime. Extensions have been written to add support for the Windows API, process management on Unix-like operating systems, multibyte strings (Unicode), cURL, and several popular compression formats. Other features include integration with IRC, dynamic generation of images and Adobe Flash content, and even speech synthesis. The language's core functions such as those dealing with strings and arrays are also implemented as an extension.^[6] The PHP Extension Community Library (PECL) project is a repository for extensions to the PHP language. PHP Data Objects (PDO) is an interface for accessing databases.^{[7][8][9][10][11]}

Zend Technologies provides a certification exam for programmers to become certified PHP developers.

Installation and configuration

There are two primary ways for adding support for PHP to a web server – as a native web server module, or as a CGI executable. PHP has a direct module interface called Server Application Programming Interface (SAPI), which is supported by many web servers including Apache HTTP Server, Microsoft IIS, Netscape (now defunct) and iPlanet. Some other web servers, such as OmniHTTPd, support the Internet Server Application Programming Interface (ISAPI), which is a Microsoft's web server module interface. If PHP has no module support for a web server, it can always be used as a Common Gateway Interface (CGI) or FastCGI processor; in that case, the web server is configured to use PHP's CGI executable to process all requests to PHP files.

PHP-FPM (FastCGI Process Manager) is an alternative FastCGI implementation for PHP, bundled with the official PHP distribution since version 5.3.3. When compared to the older FastCGI implementation, it contains some additional features, mostly useful for heavily loaded web servers.

When using PHP for command-line scripting, a PHP command-line interface (CLI) executable is needed. PHP supports a CLI SAPI as of PHP 4.3.0. The main focus of this SAPI is developing shell applications using PHP. There are quite a few differences between the CLI SAPI and other SAPIs, although they do share many of the same behaviors.

PHP can also be used for writing desktop graphical user interface (GUI) applications, by using the PHP-GTK extension. PHP-GTK is not included in the official PHP distribution, and as an extension it can be used only with PHP versions 5.1.0 and newer. The most common way of installing PHP-GTK is compiling it from the source code.

When PHP is installed and used in cloud environments, software development kits (SDKs) are provided for using cloud-specific features. For example:

- Amazon Web Services provides the AWS SDK for PHP
- Windows Azure can be used with the Windows Azure SDK for PHP.

Numerous configuration options are supported, affecting both core PHP features and extensions. Configuration file `php.ini` is searched for in different locations, depending on the way PHP is used. The configuration file is split into various sections, while some of the configuration options can be also set within the web server configuration.

Use

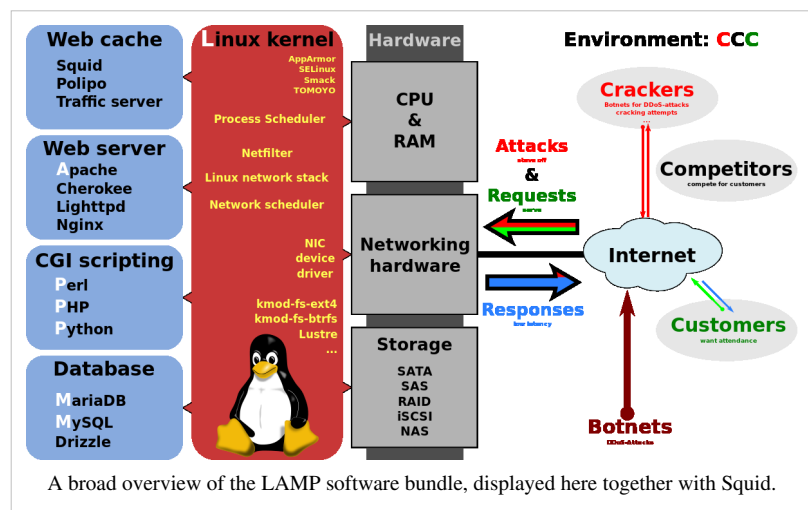
PHP is a general-purpose scripting language that is especially suited to server-side web development where PHP generally runs on a web server. Any PHP code in a requested file is executed by the PHP runtime, usually to create dynamic web page content or dynamic images used on websites or elsewhere. It can also be used for command-line scripting and client-side graphical user interface (GUI) applications. PHP can be deployed on most web servers, many operating systems and platforms, and can be used

with many relational database management systems (RDBMS). Most web hosting providers support PHP for use by their clients. It is available free of charge, and the PHP Group provides the complete source code for users to build, customize and extend for their own use.

PHP acts primarily as a filter, taking input from a file or stream containing text and/or PHP instructions and outputting another stream of data. Most commonly the output will be HTML, although it could be JSON, XML or binary data such as image or audio formats. Since PHP 4, the PHP parser compiles input to produce bytecode for processing by the Zend Engine, giving improved performance over its interpreter predecessor.

Originally designed to create dynamic web pages, PHP now focuses mainly on server-side scripting, and it is similar to other server-side scripting languages that provide dynamic content from a web server to a client, such as Microsoft's ASP.NET, Sun Microsystems' JavaServer Pages, and `mod_perl`. PHP has also attracted the development of many software frameworks that provide building blocks and a design structure to promote rapid application development (RAD). Some of these include PRADO, CakePHP, Symfony, CodeIgniter, Laravel, Yii Framework, and Zend Framework, offering features similar to other web application frameworks.

The LAMP architecture has become popular in the web industry as a way of deploying web applications. PHP is commonly used as the *P* in this bundle alongside Linux, Apache and MySQL, although the *P* may also refer to Python, Perl, or some mix of the three. Similar packages, WAMP and MAMP, are also available for Windows and OS X, with the first letter standing for the respective operating system. Although both PHP and Apache are provided as part of the Mac OS X base install, users of these packages seek a simpler installation mechanism that can be more easily kept up to date.



As of April 2007[2], over 20 million Internet domains had web services hosted on servers with PHP installed and `mod_php` was recorded as the most popular Apache HTTP Server module. As of October 2010[2], PHP was used as the server-side programming language on 75% of all websites whose server-side programming language was known (as of February 2014[2], the percentage had reached 82%), and PHP was the most-used open source software within enterprises. Web content management systems written in PHP include MediaWiki, Joomla, eZ Publish, SilverStripe, WordPress, Drupal, Moodle, the user-facing portion of Facebook, and Digg.

For specific and more advanced usage scenarios, PHP offers a well defined and documented way for writing custom extensions in C or C++. Besides extending the language itself in form of additional libraries, extensions are providing a way for improving execution speed where it is critical and there is room for improvements by using a true compiled language. PHP also offers well defined ways for embedding itself into other software projects. That way PHP can be easily used as an internal scripting language for another project, also providing tight interfacing with the project's specific internal data structures.

PHP received mixed reviews due to lacking support for multithreading at the core language level, though using threads is made possible by the "pthreads" PECL extension.

Security

In 2013, 9% of all vulnerabilities listed by the National Vulnerability Database were linked to PHP; historically, about 30% of all vulnerabilities listed since 1996 in this database are linked to PHP. Technical security flaws of the language itself or of its core libraries are not frequent (22 in 2009, about 1% of the total although PHP applies to about 20% of programs listed). Recognizing that programmers make mistakes, some languages include taint checking to automatically detect the lack of input validation which induces many issues. Such a feature is being developed for PHP, but its inclusion into a release has been rejected several times in the past.

There are advanced protection patches such as Suhosin and Hardening-Patch, especially designed for web hosting environments.

There is a number of language features and historical configuration parameters that make PHP prone to security issues:

- `magic_quotes_gpc` and `register_globals` configuration directives are the best known; the latter made any URL parameters become PHP variables, opening a path for serious security vulnerabilities by allowing an attacker to set the value of any uninitialized global variable and interfere with the execution of a PHP script. Support for "magic quotes" and "register globals" has been deprecated as of PHP 5.3.0, and removed as of PHP 5.4.0.
- implied type conversions that results in incompatible values being treated as identical against the programmer's intent; for example, comparison of `0e1234 == 0` will be *True* because the first value will be treated as scientific notation number 0×10^{1234} with value of 0. This feature resulted in authentication vulnerabilities in Simple Machines Forum, Typo3 and phpBB when MD5 password hashes were compared. Instead, identity operator (`===`) should be used: `0e1234 === 0` will result in *False*.

Another example for the runtime settings vulnerability comes from failing to disable PHP execution (via `engine` configuration directive) for the directory where uploaded images are stored; leaving the default settings can result in execution of malicious PHP code embedded within the uploaded images. Also, leaving enabled the dynamic loading of PHP extensions (via `enable_dl` configuration directive) in a shared web hosting environment can lead to security issues.

References

- [1] <http://www.php.net>
- [2] <http://en.wikipedia.org/w/index.php?title=PHP&action=edit>
- [3] PHP Manual: Preface (<http://www.php.net/manual/en/preface.php>), www.php.net
- [4] , cited at
- [5] <http://wiki.php.net/rfc/closures>
- [6] Cross Reference: /PHP_5_4/ext/standard/ (http://lxr.php.net/xref/PHP_5_4/ext/standard/)
- [7] O'Reilly Networks - ONLamp (<http://www.onlamp.com/pub/a/php/2004/08/05/dataobjects.html>)
- [8] IBM DeveloperWorks (<http://www-128.ibm.com/developerworks/db2/library/techarticle/dm-0612xia/>)
- [9] IBM Redbooks (<http://www.redbooks.ibm.com/abstracts/sg247218.html>)
- [10] phplarchitect (<http://www.phparch.com/issue.php?mid=65>)
- [11] Info (http://www.infoworld.com/article/05/10/19/HNphpshow_1.html)

External links

- Official website (<http://www.php.net>)
 - PHP (<http://www.dmoz.org/Computers/Programming/Languages/PHP>) at DMOZ
 - PHP Reference Manual (<http://www.php.net/manual>)
 - PHP source code repository @ Github (<https://github.com/php/php-src>)
 - Salary of a PHP Engineer (<http://www.theengineersalary.com/2014/04/php-engineer-salary-how-much-does-php.html>) The Engineer Salary
-

Two Bananas, Txt.file, Txuspe, Tychay, Tylaw, Typer85, UU, Udbhavah, Ugog Nizdast, Uncle G, UncleDouggie, Uncomplicated007, Unknown W. Brackets, Unreal128, Untitledmind72, Uranther, Urdutext, Uris, User24, Utcursch, UtherSRG, Uzume, VShaka, VX, Vague Rant, Val42, Valugi, Vameza, Vanished user kjj32ro9j4tkse, Vasya, Veinor, Verminox, Versageek, Versus22, Verycuriousboy, Vicki Rosenzweig, Vicky191286, Vindicator26, Violetriga, Vishal K M, Visor, Vladkornea, Vrenator, Wackyvorlon, WalterGR, Wasted Time R, Wdflake, Weathereye, WebDome, Wenz, Wesley, Weyes, Weylinp, Wez.p., Wgw2024, Whale plane, Who, Whpq, Wickorama, Wifefox, Wik, WikHead, Wiki alf, WikiLaurent, Wikibofh, Wikitanvir, Wildhoney25, Willhsmiit, Wilsonge, Wimg, Winterst, Wizardman, Wjgilmore, Wllm, Wonko, Woohookitty, Wootery, Writ Keeper, WriterHound, Wtshymanski, Wxop, Wykis, X-Fi6, XDanielt, Xantorohara, Xophorus, Xtremejames183, Xxpor, Yacoubean, Yamla, Yannick56, Yintan, Ynhockey, Yoghurt, YourEyesOnly, Yurik, Yywin, Zachlipton, Zae, Zantolak, Zawersh, Zaxx81, Zcahmed, Zenohockey, Zerbu, ZeroOne, Zeroasterisk, Zeus, ZimZalaBim, Zimbabwer, Zoe, Zoef1234, Zoicon5, Zondor, Zoonosis, Zootm, ZorkFox, Zotag, Zundark, Zvn, Zzuuzz, ^demon, Ævar Arnfjörð Bjarmason, Σ, 石, 3645 anonymous edits

Image Sources, Licenses and Contributors

File:PHP-logo.svg *Source:* <http://en.wikipedia.org/w/index.php?title=File:PHP-logo.svg> *License:* Public Domain *Contributors:* Biktora, Chealer, Pixewakb, 1 anonymous edits

file:Wikibooks-logo-en-noslogan.svg *Source:* <http://en.wikipedia.org/w/index.php?title=File:Wikibooks-logo-en-noslogan.svg> *License:* Creative Commons Attribution-Sharealike 3.0 *Contributors:* User:Bastique, User:Ramac et al.

file:Rasmus Lerdorf cropped.jpg *Source:* http://en.wikipedia.org/w/index.php?title=File:Rasmus_Lerdorf_cropped.jpg *License:* Creative Commons Attribution-Sharealike 2.0 *Contributors:* Rasmus_Lerdorf_2003.jpg: Sebastian Bergmann (Sebastian Bergmann @ Flickr) derivative work: Austin512 (talk)

file:Andi Gutmans 1.jpg *Source:* http://en.wikipedia.org/w/index.php?title=File:Andi_Gutmans_1.jpg *License:* Creative Commons Attribution 2.0 *Contributors:* jim Winstead from los angeles, usa

file:Zeev Suraski 2005 cropped.jpg *Source:* http://en.wikipedia.org/w/index.php?title=File:Zeev_Suraski_2005_cropped.jpg *License:* Creative Commons Attribution-Sharealike 2.0 *Contributors:* Zeev_Suraski_2005.jpg: Christophe Gesché (Moosh Be @ Flickr) derivative work: Austin512 (talk)

File:LAMP software bundle.svg *Source:* http://en.wikipedia.org/w/index.php?title=File:LAMP_software_bundle.svg *License:* Creative Commons Attribution-Sharealike 3.0 *Contributors:* User:ScotXW

License

Creative Commons Attribution-Share Alike 3.0
[//creativecommons.org/licenses/by-sa/3.0/](http://creativecommons.org/licenses/by-sa/3.0/)